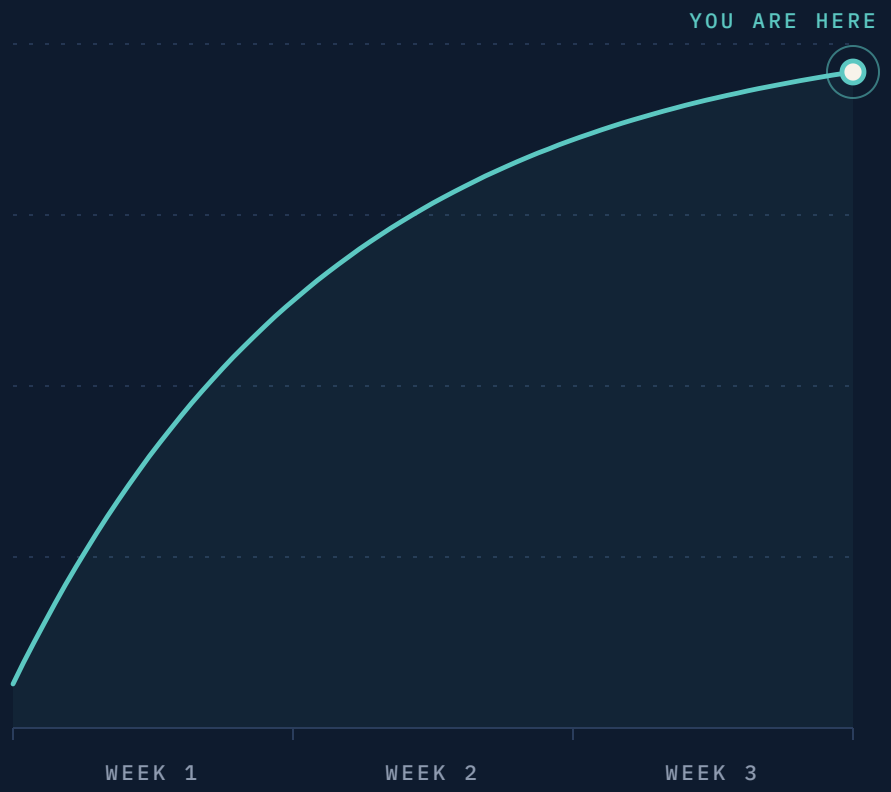
WEEK 3 · DAY 5 · OVERVIEW

Course 5 Capstone



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 12 SECTIONS / 3 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

A colleague who covers Asian corporates hands you a public file of Taiwanese companies' financial ratios, with a flag for the ones that went bankrupt.

The dataset page says the data cover 1999 to 2009 and bankruptcy follows the Taiwan Stock Exchange's regulations, with no units, no horizon and no company identifier, so this notebook speaks of rows.

It is one market in one period, and that market differs from US credit.

She asks which of four models best separates bankrupt rows it never saw, how sure 44 bankrupt test rows let anyone be, and what the chosen model leans on.

Its output is "the model's probability of bankruptcy in this file", never a credit view on any company.

The work is yours: each question has an empty cell and a **check()** line.

OBJECTIVES

Describe a new file's columns, copies and scales, and split it once.

Compare four models on folds, tune the leader honestly, and choose weights, resampling and a threshold out of fold.

Open the test rows once, put an interval on each score, and describe the chosen model with SHAP values on rows it never saw.

ABOUT 55 MIN WITH EXERCISES / 12 SECTIONS / 3 EXERCISES

Your Route Through Day 5

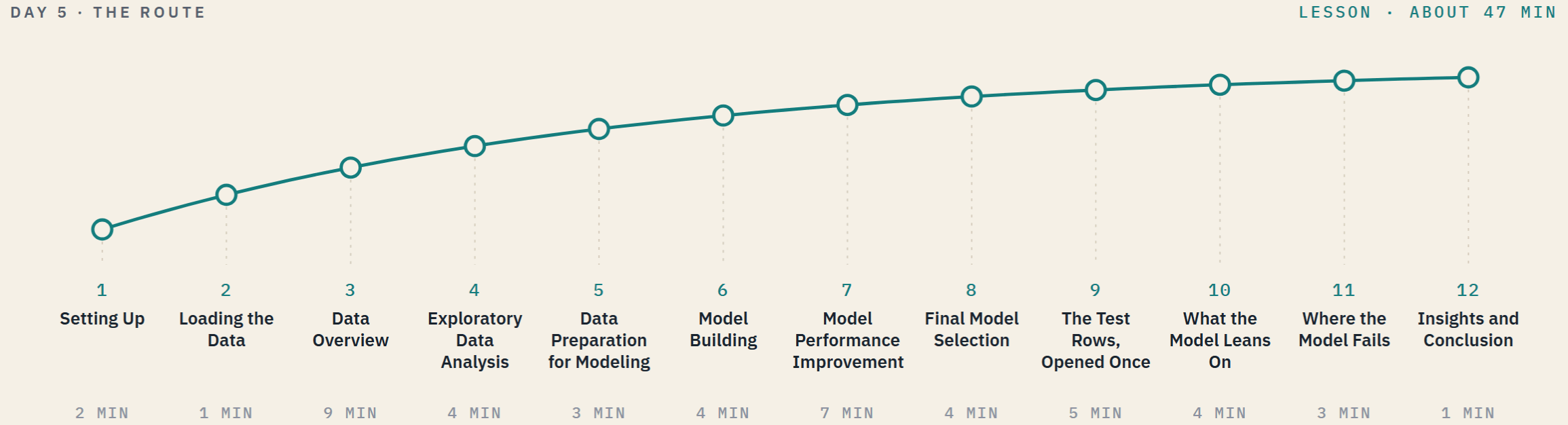


FIGURE 2 Twelve sections, in order, with about how long each takes.

Time. About 47 minutes for the lesson, then about 9 for the three exercises.

Setting Up

THE ANSWER CHECKER

```
# check() compares a fingerprint of your answer with a stored one, so no answer is written on the page
import hashlib
def check(label, answer, code, places=None):
    if answer is ...:
        print(label + ": not attempted yet")
        return
    # a number is rounded to the places the question asks for, so 0.66241 and 0.6624 read the same
    if places is not None:
        answer = round(float(answer), places)
    text = label + "|" + str(answer).strip().lower()
    # hashlib.sha256() makes a fingerprint that cannot be turned back into the answer;
    # .encode() turns the text into bytes for it, and .hexdigest() writes it as letters and digits
    if hashlib.sha256(text.encode()).hexdigest()[:12] == code:
        print(label + ": correct")
    else:
        print(label + ": not yet. You have " + str(answer))
```

`check()` **compares a fingerprint of each answer with a stored one, so no answer is written on the page.**

Read the file and clean its names.

Loading the Data

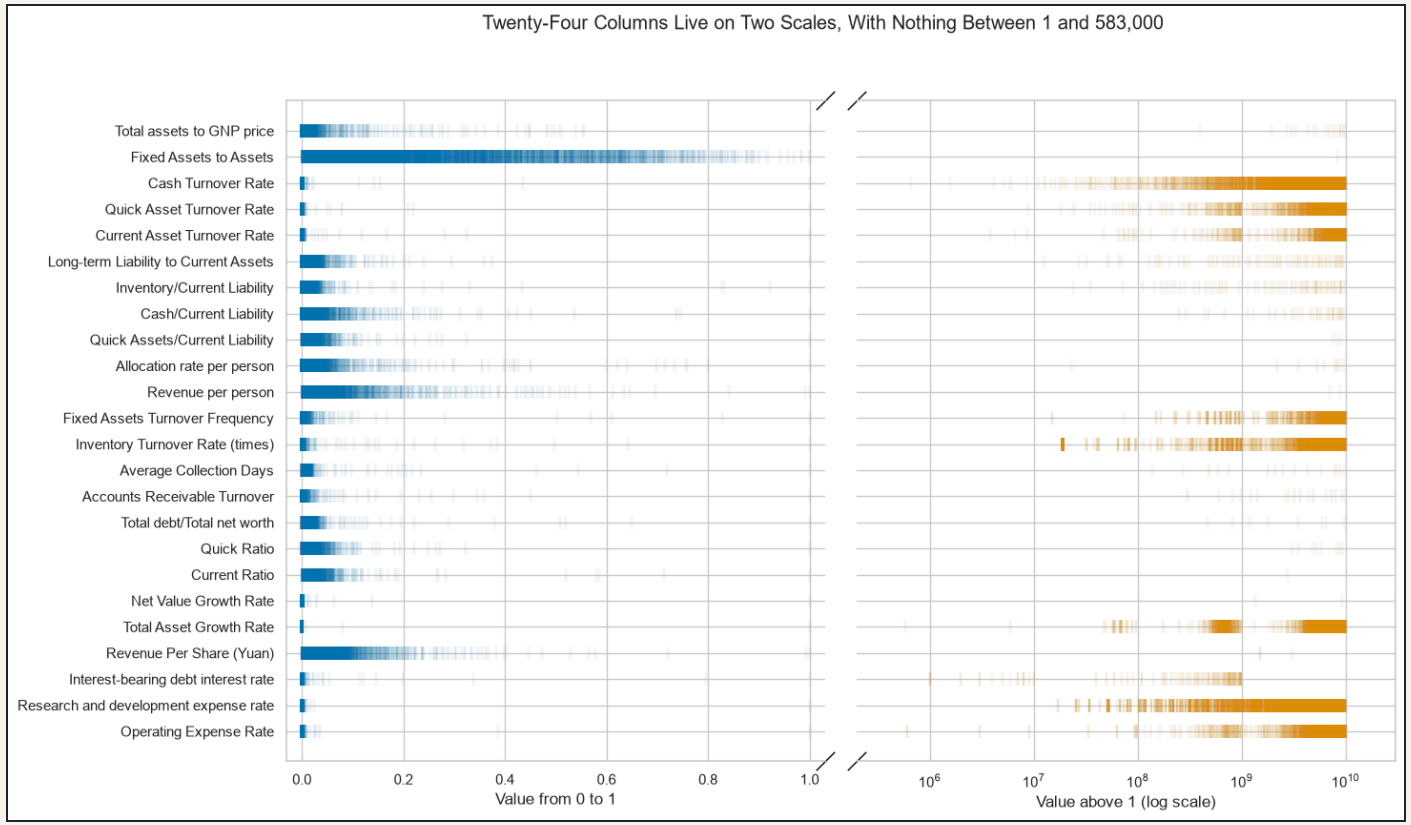
READ THE FILE AND CLEAN THE NAMES

```
# the 6,819 rows: the outcome first, then 95 features whose names start with a space
ratios = pd.read_csv(DATA + "taiwan_bankruptcy.csv")
# each name without its leading space, and without the space and yen sign that end three names
ratios.columns = [name.strip().replace(" \u00a5", "") for name in ratios.columns]
TARGET = "Bankrupt?"
FEATURES = [name for name in ratios.columns if name != TARGET]
```

THE COLUMNS

- `Bankrupt?`: 1 if the company went bankrupt, **the target**
- 95 features under their own names; the page's X1 to X95 list runs in another order, so it is quoted nowhere as a definition

Data Overview



WHAT THE SECTION ASKS

WHAT IT ASKS
How many pairs of features correlate above 0.99 in absolute value, how many of them are equal in every row, and how many features do they hold?
Draw one for the near pairs, the equal pairs dark.
How many features hold a value above 1?
For `Total Asset Growth Rate` and `Cash Turnover Rate`, what percent of rows is bankrupt above 1, and at or below 1?
Draw the four as dots with their margins as whiskers, the whole file's share dashed.

One constant, two equal pairs and two scales: the preparation drops three columns and learns any list it needs from the training rows.

Count bankrupt rows by tenth and by debt.

Exploratory Data Analysis

QUESTION	WHAT IT ASKS
Predict 1	Sketch the ten counts you expect.
The Order of the File	Count the bankrupt rows in each tenth as a list of plain numbers (`int(...)` makes one) in **`q7_tenths`**, and store the position of the first operating row in **`q7_first_operating`**.
Debt by Outcome	What is the median **`Debt ratio %`** of the bankrupt rows, and of the others?
	Draw each outcome's **`Debt ratio %`** as an ECDF (Course 4's Day 6).

TEN SLICES

Cut the file into ten slices in its own row order.

THE FUNCTION THAT CUTS

`np.array_split(values, 10)` cuts an array into ten slices, in order.

Data Preparation for Modeling

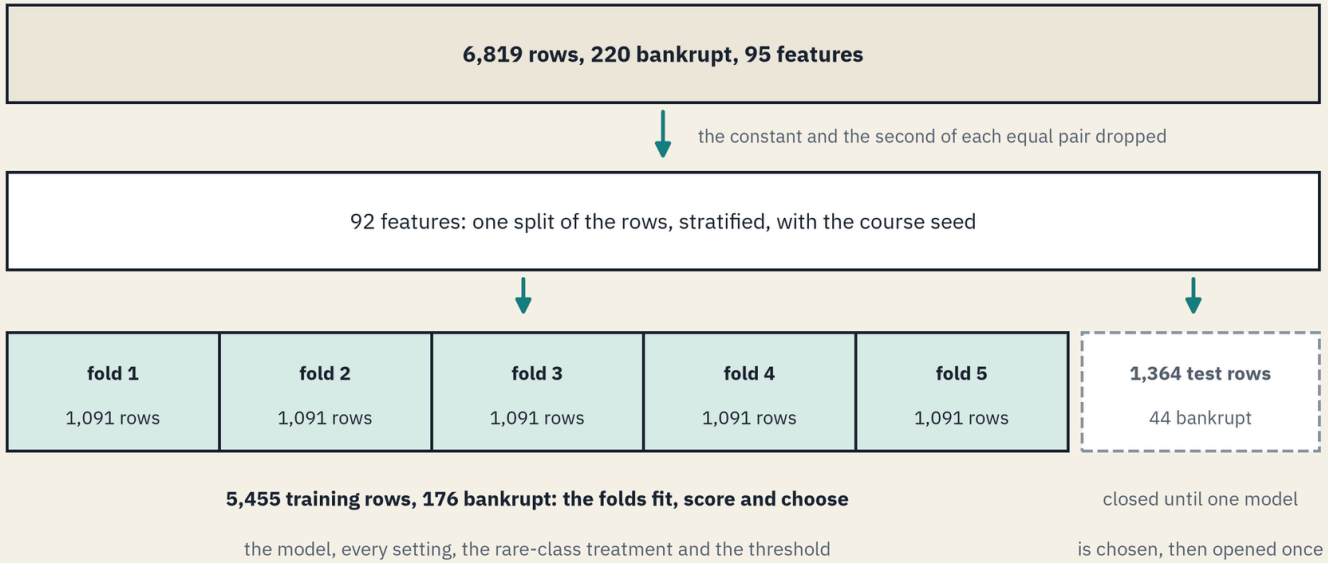


FIGURE 3 6,819 rows, three columns dropped, 5,455 training rows in five folds that choose, 1,364 test rows opened once.

WHAT THE SECTION ASKS

WHAT IT ASKS
Drop the constant and the second feature of each equal pair, then split 80/20 into `X_train, X_test, y_train, y_test` with `stratify=y` and the course seed.
Keep the features above 1 in the training rows in a list `two_scale`.

Every choice from here is made on the 5,455 training rows.

Score a logistic and three ensembles on folds.

Model Building

QUESTION	WHAT IT ASKS
Course 4's Logistic, Without Its Weights	Build **`logistic`**, a pipeline of **`QuantileTransformer(output_distribution="normal", random_state=SEED)`** and **`LogisticRegression(solver="newton-cholesky", tol=1e-8)`** (steps **`"quantile"`**, **`"logit"`**), and score it with **`cross_validate`** on **`folds`**, scoring **`["average_precision", "roc_auc"]`**.
Three Ensembles	Put four models in a dictionary **`models`** under **`"logistic"`**, **`"forest"`** (Day 3's random forest, here 100 trees with leaves of 5 rows or more), **`"xgboost"`** (XGBoost with two of Day 8's brakes, depth 3 and a step of 0.05, over 300 trees) and **`"lightgbm"`** (Day 11's LightGBM at its defaults), each with the seed and **`n_jobs=1`**.
	Draw each fold as a line across the models, the means in bold.

THE SCORING FUNCTION

```
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
def threshold_for_recall(target, output, target_recall=0.70):
    precision, recall, thresholds = precision_recall_curve(target, output)
    return thresholds[recall[:-1] >= target_recall].max()
```

Model Performance Improvement

QUESTION	WHAT IT ASKS
Indicators for the Two Scales	Write **`add_above_one(frame)`**, returning the frame plus a 0/1 column per feature in **`two_scale`**, 1 above 1.
A Random Search for LightGBM	Search Day 11's space for **`models["lightgbm"]`** (**`num_leaves`** from **`randint(8, 32)`**, **`learning_rate`** from **`loguniform(0.03, 0.3)`**) with **`RandomizedSearchCV(..., n_iter=6, scoring="average_precision", cv=folds, random_state=SEED, n_jobs=1)`**, as **`search`**.
	Draw Day 11's search-space scatter, the defaults (31 leaves, 0.1) as a cross.
Predict 2	Run the whole search inside each fold, as Day 11 did, and score each winner on the fold it never saw.
The Search, Scored Honestly	Score **`inner_search`**, the same search on **`StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)`**, with Day 11's nested **`cross_validate(..., cv=folds, return_estimator=True, n_jobs=1)`**.
Weights, SMOTE, or Neither	Score LightGBM two more ways on **`folds`**, both Day 13's: **`class_weight="balanced"`**, and **`SMOTE(random_state=SEED)`** before it in an **`ImbPipeline`** (steps **`"smote"`**, **`"lightgbm"`**).
	Draw the means as bars, the fold scores as dots.
	Score each version out of fold with **`cross_val_predict(..., method="predict_proba")`** and read its threshold for a recall of 0.70.

Set thresholds and choose one model on folds.

Final Model Selection

QUESTION	WHAT IT ASKS
Thresholds for a Recall of 0.70	Keep the four models' out-of-fold outputs in a dictionary **`out_of_fold`** (LightGBM's are **`outputs["neither"]`**) and their thresholds for 0.70 in **`thresholds`**.
	Draw the four out-of-fold precision-recall curves, recall 0.70 dashed, each threshold a dot.
Judged on Their Own Rows	Fit each model on all the training rows and score it there.
	Draw each model as a dumbbell from its fold mean to its training-row score.
The Choice	Which model goes forward, by the criterion set before any model was fitted?

One model goes forward, chosen before one test row is scored.

The Test Rows, Opened Once

QUESTION	WHAT IT ASKS
Predict 3	How many of the 44 test bankruptcies will it flag?
Predict 4	Which of the four models will have the highest average precision on the test rows, and what will it be?
The Test Table	Score the four models on the test rows in one table **`test_comp`**, each at its threshold, and keep their test outputs in a dictionary **`test_outputs`**.
	Draw the four confusion matrices in Course 4's colours.
Predict 5	Resample the 1,364 test rows 1,000 times.
How Sure Can Anyone Be?	With Day 10's **`rng = np.random.default_rng(SEED)`** and **`draws = rng.choice(1364, size=(1000, 1364))`**, score each model's test average precision on every resample, in a table **`boot_ap`**.
	Draw one, with the test scores, their intervals and the fold means as rings on the left, and LightGBM minus each model as a half violin with its interval as a bar on the right.
Which Bankruptcies Each Model Catches	For each test bankruptcy, note which models flag it, in a table **`caught`**, one column per model.

What the Model Leans On

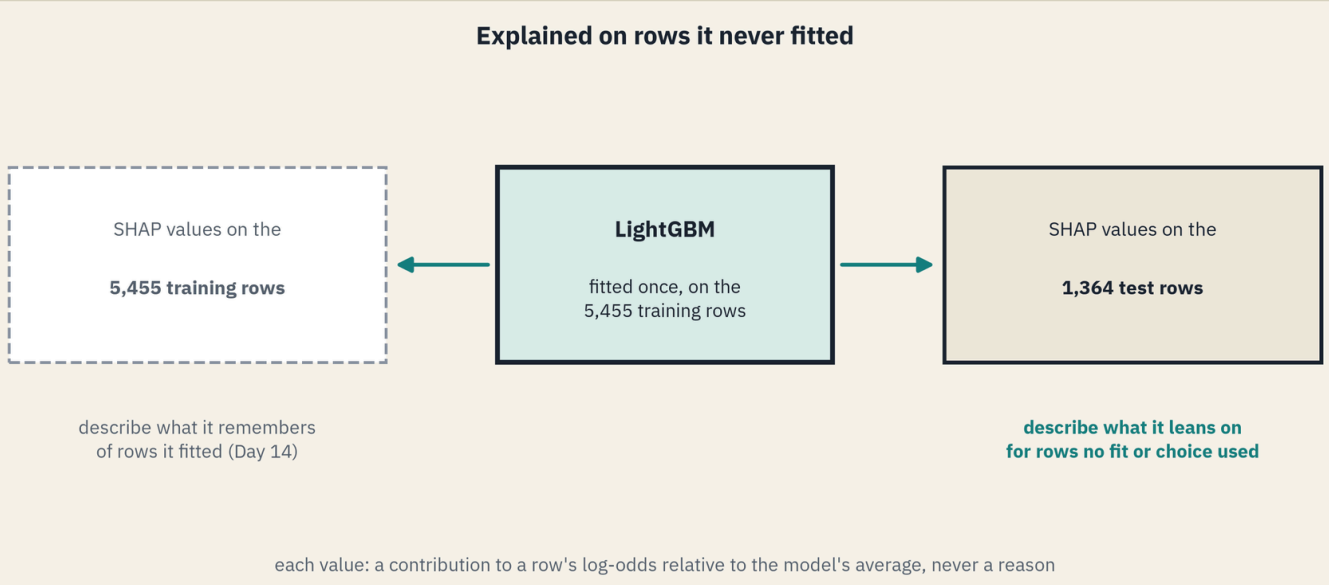


FIGURE 4 LightGBM fitted on the 5,455 training rows, its SHAP values read on the 1,364 test rows it never fitted.

WHAT THE SECTION ASKS

WHAT IT ASKS
Explain the fitted LightGBM on `X_test` with Day 14's `shap.TreeExplainer`, as `shap_test`.
Draw the beeswarm of the top ten with `shap.summary_plot(..., max_display=10, rng=SEED, show=False)`.
Keep LightGBM's split counts, Day 9's `booster_.feature_importance("split")`, in a Series `splits`.
Draw a slopegraph from rank by split count to rank by mean absolute SHAP value for the features in either top ten, two-scale features in colour.

Find the misses and add up the probabilities.

Where the Model Fails

QUESTION	WHAT IT ASKS
The Misses	Which test bankruptcies does LightGBM miss at its threshold? Store how many, the highest probability among them (5 decimals) and its row label in **`q26_missed`**, **`q26_highest`** and **`q26_row`**, and compare the median **`Total debt/Total net worth`** of the missed, caught and operating test rows.
	Draw that row's waterfall (Day 14).
What the Probabilities Add Up To	Add up LightGBM's out-of-fold probabilities on the training rows, and its probabilities on the test rows. Store the sums in **`q27_oof`** and **`q27_test`** (1 decimal), against 176 and 44 bankruptcies.

READING A WATERFALL

A SHAP value is a contribution to a row's log-odds relative to the model's average, never a reason.

WHY THE TEST ROWS

Day 14 found that SHAP values on fitted rows describe memory, so the chosen model is described on the 1,364 test rows it never fitted.

Insights and Conclusion

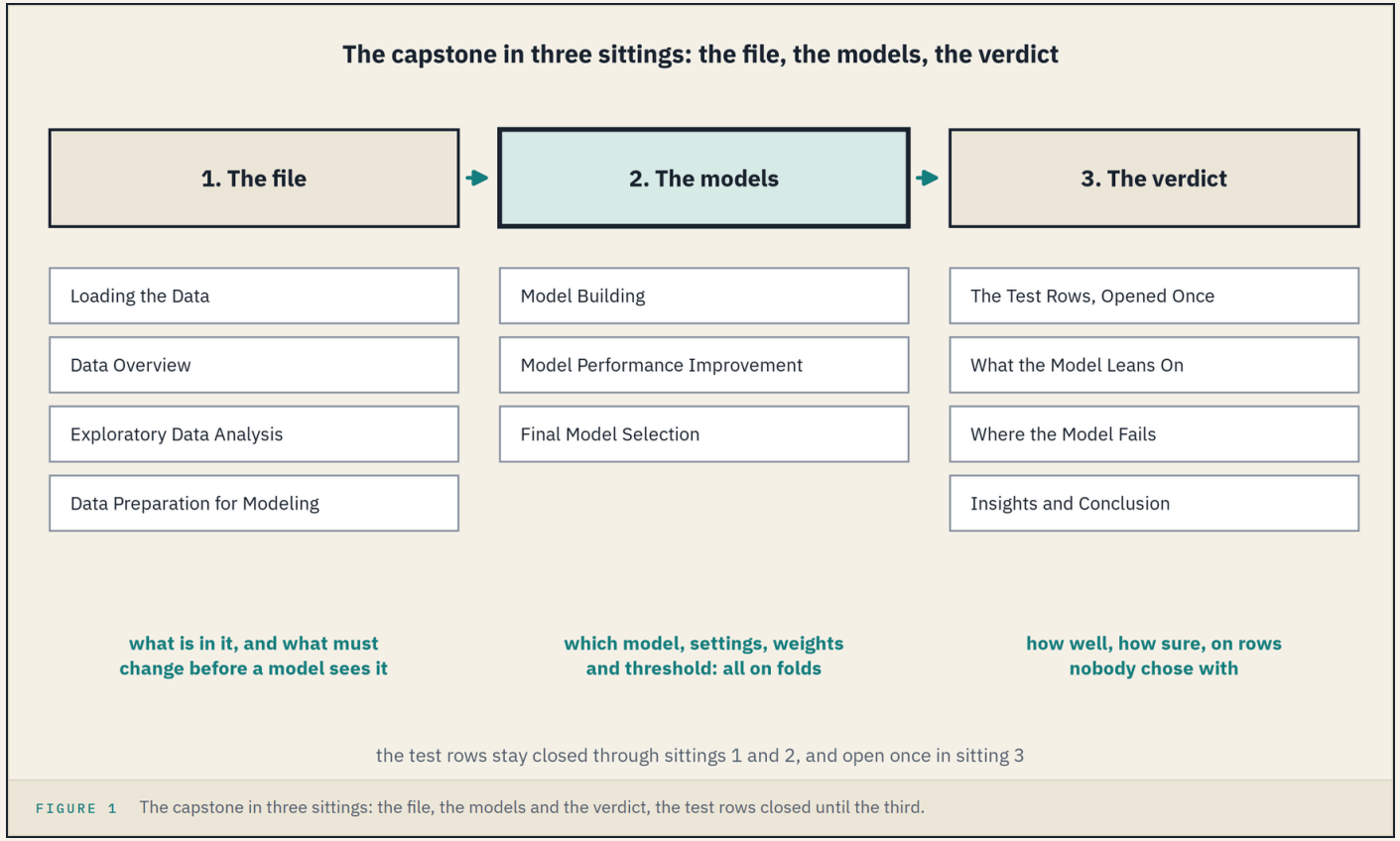


FIGURE 1 The capstone in three sittings: the file, the models and the verdict, the test rows closed until the third.

WHAT THE SECTION ASKS

WHAT IT ASKS
Write **`description`**, a few sentences in one string: how well the chosen model ranks the test rows and how sure anyone can be, where it is wrong, what the data cannot support, and what was not tested.

A model describes the file it was fitted on.

You Can Now

- Describe a new file's constants, copies, near-copies and scales before any model sees it.
- Compare models on folds and score a search with nested cross-validation.
- Choose class weights, resampling and a threshold from out-of-fold outputs.
- Open the test rows once and put an interval, paired or not, on each score.
- Describe a model with SHAP values on rows it never fitted.

NEXT
WEEK 1 · DAY 1

Missing Data, Gaps, and Corrections

Course 5B turns to dated series, starting with the gaps in Treasury yields.